

ACTION-REQUIRED

ACTION REQUIRED — the manual-step system

The wizard ends with a section it cannot fake:

```
=====
ACTION REQUIRED – 3 step(s) only you can do
=====
A shell script cannot run Claude Code slash commands or use your
sudo.
These are NOT done. Nothing above claims they are.

1. Activate the ashlr plugin inside Claude Code
  /plugin marketplace add ashlr/ashlr-plugin
  /plugin install ashlr@ashlr-marketplace
  /reload-plugins
  /ashlr:ashlr-status          # confirm it is live

2. Build the Lumen semantic index for this project
  ./scripts/lumen-reindex.sh "/path/to/project"
  ./scripts/lumen-index-doctor.sh "/path/to/project" # verify
afterwards

3. Refresh PATH in your existing terminals
  exec bash -l          # or just open a new terminal

  Saved to /path/to/project/MANUAL-STEPS.md so you can come back
to it.

Read the 3 step(s) above. Press Enter to continue...
```

Why it exists

Some work is genuinely outside a shell script’s reach:

- **Claude Code slash commands** (/plugin marketplace add, /plugin install, /reload-plugins, /ashlr:ashlr-genome-init) run *inside* a Claude Code session. There is no CLI equivalent a bash script can call.
- **Privileged host changes** need the operator’s sudo. This machine has no passwordless sudo, and even where it exists, restarting a system service is not a decision an installer should make silently.

The failure mode this replaces is the one that matters: a wizard that skips such steps quietly and then prints a green summary, so **work that was never done reads as done**. The section above states the opposite explicitly — *“These are NOT done. Nothing above claims they are.”*

Tests A48 (the section exists) and A49 (the steps are persisted to a file) keep it there.

The detection trick: cache/ vs marketplaces/

This is the whole mechanism. Claude Code keeps two different directories under its config dir, and they mean two different things:

Path	Written by	Means
<claude-dir>/ plugins/cache/ <marketplace>/ <plugin>/	the vendor’s install.sh, run by the wizard	The bits are on disk. Nothing is wired into Claude Code.
<claude-dir>/ plugins/ marketplaces/ <marketplace>/	Claude Code itself, when the operator runs /plugin marketplace add	A human actually ran the slash command. The plugin is registered.

So the wizard can distinguish “I downloaded it” from “you activated it” without asking, and without a state file of its own:

```
CLAUDE_DIR="${CLAUDE_CONFIG_DIR:-$HOME/.claude}"
ASHLR_CACHE="$CLAUDE_DIR/plugins/cache/ashlr-marketplace/ashlr"
ASHLR_MKT="$CLAUDE_DIR/plugins/marketplaces/ashlr-marketplace"
```

```

if  [[ -d "$ASHLR_CACHE" && ! -d "$ASHLR_MKT" ]]; then  #
    cloned, not activated
    → "Activate the ashlr plugin inside Claude Code"
elif [[ -d "$ASHLR_MKT" && ! -d "$PROJECT_ROOT/.ashlrcode/
    genome" ]]; then
    → "Initialise the ashlr genome for this project"
fi

```

The detection is **stateful and self-clearing**. Do the step, re-run the wizard, and the entry disappears — because the evidence of having done it is a directory that now exists. Nothing has to be ticked off, and nothing can drift out of sync with reality. Test A50 asserts the plugins/marketplaces/ashlr-marketplace probe specifically, because checking only the cache directory would report the plugin as done the moment it was downloaded.

Note the asymmetry in the current code. The CLAUDE_CONFIG_DIR override is honoured in this detection block, but Step 2's install check and the final summary both hardcode \$HOME/.claude/plugins/cache/ashlr-marketplace/ashlr. On a host running with a non-default CLAUDE_CONFIG_DIR, those two can disagree with the ACTION REQUIRED section. Trust the ACTION REQUIRED section — it is the one that reads the active config dir.

Every step the wizard can raise

In the order they are emitted. All conditions are re-evaluated on every run.

The first two are registered **during** the run — Step 7 and Step 9 — rather than in the drain block at the end, so they appear first in the list and first in MANUAL-STEPS.md.

1. Apply the ollama concurrency setting computed for THIS host

When: scripts/ollama-tune.sh produced commands and the wizard did not apply them — which is the default, and remains the case when WIZARD_TUNE_OLLAMA is unset, when an indexer is in flight, when the indexer state cannot be determined, when you decline at the prompt, or when an attempted --apply failed.

The body is **the delegate's own --print-commands output for this machine**, followed by the opt-in and the undo:

```
<the exact commands ollama-tune.sh generated for this host>

# Or let the wizard do it - it asks first and refuses while an
indexer runs:
#   WIZARD_TUNE_OLLAMA=1 ./scripts/setup-agents-wizard.sh
# Undo at any time:
#   ./scripts/ollama-tune.sh --revert
#
# WHY THIS IS NOT AUTOMATIC: applying RESTARTS the ollama service,
which aborts
# every in-flight embedding request. An index run in progress
loses that work.
```

No example is ever substituted. If the tuner is absent, fails, or ollama is not there at all, the step still appears — but it says that nothing was measured and offers the tuner, rather than printing a plausible number the wizard did not compute. Test K2b asserts that the absent-tuner step contains no concurrency value at all, and K5b asserts that the normal step carries the token the delegate generated on this run.

2. Provider-side CI triggering

When: scripts/verify-provider-ci.sh exited 1 (confirmed), or the status could not be determined — exit 2, a timeout, an unexpected status, a missing script, or a missing gh.

Two distinct bodies, and they must not be confused:

Verifier said	Title	Body
confirmed (1)	Provider-side CI triggering was CONFIRMED just now - operator-only to change	The verifier’s own findings, verbatim and timestamped, plus a reminder that no file in any of those repositories can turn them off — it is repo/org Settings → Actions, Pages, Branches
unknown (2, timeout, other, absent script, absent gh)	Provider-side CI status is UNVERIFIED ...	Why it could not reach a verdict, and the commands to find out (gh auth status, then the verifier)

A 0 raises **nothing** — that is the only case where silence is correct, because it is the only case where something was actually measured and came back clean. Test K16c asserts that an exit 2 never renders the 0 success line.

3. Activate the ashlr plugin

When: `plugins/cache/ashlr-marketplace/ashlr/` exists **and** `plugins/marketplaces/ashlr-marketplace/` does not.

```
/plugin marketplace add ashldai/ashlr-plugin
/plugin install ashlr@ashlr-marketplace
/reload-plugins
/ashlr:ashlr-status          # confirm it is live
```

See ashlr is a plugin, not a binary below.

4. Initialise the ashlr genome

When: the marketplace is registered (step 1 is done) **and** `<project-root>/.ashlrcode/genome/` does not exist.

```
/ashlr:ashlr-genome-init
```

Mutually exclusive with step 1 — it is the `elif` branch. You will never see both.

5. Take the GPU out of the embedding path

When: `ollama` is on `PATH` **and** its journal, scanned from the service's `ActiveEnterTimestamp`, reports `library=Vulkan` (or `vulkan`).

```
./scripts/ollama-vulkan-remediation.sh --check    # diagnosis,
read-only
./scripts/ollama-vulkan-remediation.sh --apply    # applies it,
then verifies
```

The wizard **delegates** rather than inlining the privileged commands. Test A41 asserts the wizard's own source contains no `systemctl` restart, no write under `/etc`, and no `sudo` in this path — applying the fix is `ollama-vulkan-remediation.sh`'s job, behind an explicit subcommand. The step text states why in one line: *this backend silently writes stale duplicate vectors under HTTP 200, well-formed, unit-norm, and invisible to every per-vector check.*

Both journal probes here are timeout-bounded (20 s and 10 s) so a hung `journalctl` or `systemctl` cannot stall the wizard.

6. WOZCODE has no public installer

When: `wozcode` is not on `PATH` **and** `WOZCODE_INSTALL_CMD` is unset.

```
WOZCODE_INSTALL_CMD='<your install command>' ./scripts/setup-  
agents-wizard.sh
```

7. The glyphdown hook was skipped on request

When: `WIZARD_SKIP_GLYPHDOWN_HOOK` is set.

```
# Re-run WITHOUT the flag to register it:  
./scripts/setup-agents-wizard.sh  
# It fires on EVERY tool call - enable deliberately.
```

8. Build the Lumen semantic index

When: `lumen` is on `PATH` **and** a bounded probe cannot confirm a usable index:

```
timeout 20 lumen search "x" -p "$PROJECT_ROOT" --summary -n 1  
  
./scripts/lumen-reindex.sh "<project-root>"  
./scripts/lumen-index-doctor.sh "<project-root>" # verify  
afterwards
```

The timeout is load-bearing, not decoration. `lumen search` blocks while an index run holds the embedding backend, and an unbounded probe once stalled the wizard for ten minutes. Test A47 asserts that every `lumen search` in the wizard is timeout-wrapped, by counting bounded against unbounded occurrences.

A timeout is treated as “**could not confirm**”, so the step is offered rather than skipped. You may therefore see this step on a project whose index is perfectly healthy but busy. That is the intended bias: over-suggesting a safe, idempotent step beats silently omitting a necessary one.

9. Refresh PATH in your existing terminals

Always. There is no condition on it.

```
exec bash -l      # or just open a new terminal
```

Because this one is unconditional, the ACTION REQUIRED section always has **at least one** entry and MANUAL-STEPS.md is always written.

MANUAL-STEPS.md

Written to <PROJECT_ROOT>/MANUAL-STEPS.md on **every** run, unconditionally overwriting the previous copy. Its shape:

```
# Manual steps the setup wizard cannot perform
```

```
Generated 2026-08-27T15:34:23Z by scripts/setup-agents-wizard.sh
```

```
## 1. <title>
```

```
````
```

```
<commands>
```

```
````
```

Points worth knowing:

- **It is a snapshot, not a checklist.** Nothing marks a step done. The file is regenerated from scratch each run and shrinks as conditions clear.
 - **It lands in your project root**, next to README.md. If you do not want it committed, add MANUAL-STEPS.md to .gitignore.
 - **It is not recorded in the rollback manifest.** scripts/rollback-agents-wizard.sh will not remove it; rm MANUAL-STEPS.md if you want it gone.
 - **Paths inside it are absolute**, expanded at generation time — safe to paste into any shell.
-

The pause

```
if [[ -t 0 && -z "${WIZARD_NONINTERACTIVE:-}" && $
    {#MANUAL_TITLES[@]} -gt 0 ]]; then
    read -r -p "Read the N step(s) above. Press Enter to
        continue..." _ack || true
fi
```

Three conditions, all required:

| Condition | Why |
|-----------|-----|
| -t 0 | |

| Condition | Why |
|--|--|
| | stdin is a terminal. In a pipe, CI job or <code>\ tee</code> the wizard never blocks. |
| <code>WIZARD_NONINTERACTIVE</code> unset | An explicit escape hatch for automation on a TTY. |
| at least one step | Vacuously true today — the last step, <i>Refresh PATH</i> , is unconditional. |

The `\|\| true` means an EOF on stdin is not an error under `set -euo pipefail`.

This is the wizard’s only blocking prompt. Everywhere else it is non-interactive; a `sudo` password prompt comes from `sudo` itself, not from the script. If you script the wizard, set `WIZARD_NONINTERACTIVE=1` — the section still prints and `MANUAL-STEPS.md` is still written, you simply are not asked to acknowledge it.

```
WIZARD_NONINTERACTIVE=1 ./scripts/setup-agents-wizard.sh
```

ashlr is a plugin, not a binary

The wizard used to run `check_command ashlr` and print a permanent, misleading `. There is no ashlr binary and there never will be` — `command -v ashlr` failing is correct behaviour, not a fault. Test A42 asserts the wizard no longer probes `PATH` for it; A43 asserts it verifies the plugin **directory** instead.

What the vendor installer (<https://plugin.ashlr.ai/install.sh>, run via `bash`, requires `bun`) actually does is clone into:

```
~/.claude/plugins/cache/ashlr-marketplace/ashlr/
```

It deliberately creates no binary and does not touch `settings.json`. Turning that clone into a working plugin takes three slash commands inside Claude Code, plus one to confirm:

```
/plugin marketplace add ashlr/ashlr-plugin
/plugin install ashlr@ashlr-marketplace
/reload-plugins
/ashlr:ashlr-status
```

The wizard prints these in Step 2 as information, and again in ACTION REQUIRED as a *pending step* — the second one only when plugins/
marketplaces/ashlr-marketplace/ is still missing.

The genome

Once the plugin is active, /ashlr:ashlr-genome-init builds a project genome at:

```
<project-root>/ashlrcode/genome/
```

On this repository it holds **15 sections** listed in manifest.json, across five directories:

| Directory | Sections |
|-------------|--|
| vision/ | north-star.md, architecture.md, principles.md, anti-patterns.md |
| milestones/ | current.md, backlog.md |
| strategies/ | active.md, graveyard.md, experiments.md |
| knowledge/ | decisions.md, discoveries.md, dependencies.md, architecture.md, conventions.md, workspace.md |

(knowledge/ carries six of the fifteen; manifest.json is the index, not a section, and proposals.jsonl and evolution/ are runtime state.)
manifest.json also records a generation counter and per-section token budgets, which is what lets ashlr route searches through genome RAG instead of re-reading the tree.

The genome is **optional** — the wizard labels the step “(*optional, improves routing*)” — and it lives inside your project, so it is yours to commit or ignore.

MiMo, MCP, and a claim that was wrong

Related to the same “never report work you did not do” principle, and corrected in the same pass:

The old claim: “~/.mimocode exists but exposes no documented MCP config file”, printed as a manual step telling you to add Lumen yourself.

The reality: MiMo Code **inherits** MCP servers from `~/.claude.json` — it labels them `claude:~/.claude.json` in `mimo mcp list`. There was never anything to configure by hand.

That mattered because `claude mcp add-json -s user` writes into the **active** `CLAUDE_CONFIG_DIR`, which is *not* necessarily `~/.claude.json`. On a session running with a non-default config dir, Claude Code got Lumen and every tool inheriting from the default file stayed blind to it. The wizard now mirrors the registration into `~/.claude.json` directly (`.mcpServers.lumen = {"type":"stdio","command":<wrapper>,"args":["stdio"]}`, jq-merged, backed up under component `claude`) and probes MiMo honestly with `timeout 25 mimo mcp list`.

Tests: A44 (the mirror exists), A45 (the stale “no documented MCP config” claim is gone), A46 (MiMo status is *probed*, not assumed).

Verify by hand:

```
mimo mcp list | grep lumen
jq -c '.mcpServers.lumen' ~/.claude.json
```

Related

- [OPERATIONAL-SCRIPTS.md](#) — the three scripts these steps point at.
- [OLLAMA-REMEDIATION.md](#) — the runbook behind step 3.
- [USER_GUIDE.md](#) — what the rest of the run prints.
- [MANUAL.md](#) — `manual_step`, `MANUAL_TITLES/MANUAL_STEPS`, and the environment-variable reference.